

GPUMODE Mega Lecture 91 Study Notes: Reinforcement Learning, Agents, and OpenEnv

Core Problem the Lecture Addresses

This lecture is a multi-speaker deep dive into reinforcement learning for language-model agents, with a focus on why static supervised fine-tuning is not enough, why interactive environments matter, how OpenEnv standardizes those environments, and why scalable sandbox infrastructure becomes a first-order systems problem once reinforcement learning is applied to agentic workloads.

The core technical problem is:

How do we train language models to become reliable agents that can interact with tools, games, code repositories, web browsers, APIs, and sandboxes, while receiving verifiable rewards and improving through reinforcement learning at large scale?

The lecture argues that the next stage of post-training is not merely more supervised data. It is a shift toward:

- **verifiable environments**, where the model can take actions and receive observations;
- **reward functions**, which turn behavior into optimization signals;
- **curricula**, which control task difficulty over time;
- **sandboxed execution**, which safely runs model-generated code or actions;
- **scalable infrastructure**, which can support millions of concurrent environment sessions;
- **observability**, which lets researchers detect reward hacking, rollout collapse, and degenerate behavior.

The recurring theme is that reinforcement learning is powerful but inefficient. It can convert weak scalar reward signals into model behavior, but only when the environment is well-specified, the probability of success is nonzero, and the training system is designed to prevent reward hacking.

Required Background

Language Model Post-Training Pipeline

A modern large language model is usually trained in several phases:

1. **Pre-training**: predict next tokens over massive corpora.
2. **Supervised fine-tuning**: imitate high-quality demonstrations.
3. **Preference fine-tuning**: align responses using comparisons, such as direct preference optimization.
4. **Reinforcement learning**: optimize behavior against rewards from models, verifiers, unit tests, games, or environments.

A simplified resource allocation model from the lecture is:

$$\text{Compute Budget} = C_{\text{pretrain}} + C_{\text{SFT}} + C_{\text{RL}}.$$

Daniel’s practical intuition was that most compute should still be spent on pre-training, because reinforcement learning becomes much more effective when the base model already has useful latent capabilities. A rough allocation was described as:

$$C_{\text{pretrain}} \gg C_{\text{SFT}} \approx C_{\text{RL}}.$$

For small or middle-class research settings, a possible heuristic is:

$$C_{\text{pretrain}} : C_{\text{SFT}} : C_{\text{RL}} \approx 80 : 10 : 10,$$

while for frontier-scale regimes, the true ratio can be closer to:

$$C_{\text{pretrain}} : C_{\text{posttrain}} \approx 99 : 1.$$

The exact ratio is not a law. The deeper principle is that reinforcement learning is a post-training amplifier, not a replacement for giving the model enough capability through pre-training and supervised data.

Agent-Environment Loop

The central abstraction in reinforcement learning is the loop:

$$\text{Agent} \rightarrow \text{Action} \rightarrow \text{Environment} \rightarrow \text{Observation, Reward} \rightarrow \text{Agent}.$$

At time step t , the agent observes state information o_t , chooses an action a_t , and receives a reward r_t :

$$a_t \sim \pi_\theta(a_t \mid h_t),$$

where π_θ is the policy represented by the language model and h_t is the history of messages, observations, and previous actions. The environment returns:

$$(o_{t+1}, r_t, d_t) = E(o_t, a_t),$$

where d_t indicates whether the episode is done.

For a language-model agent, the policy is a token-generating model:

$$\pi_\theta(y \mid x) = \prod_{i=1}^T p_\theta(y_i \mid x, y_{<i}).$$

The action may be a word guess, a code snippet, a tool call, a shell command, a browser action, a game move, or an API call.

Rollouts

A rollout is a sampled trajectory from the current policy:

$$\tau = (x, y_1, y_2, \dots, y_T, r),$$

or, in an interactive environment:

$$\tau = (o_0, a_0, r_0, o_1, a_1, r_1, \dots, o_T).$$

The total return is often:

$$R(\tau) = \sum_{t=0}^T \gamma^t r_t,$$

where γ is a discount factor. For many language-model tasks, the reward may be sparse and given only at the end:

$$R(\tau) = r_T.$$

This sparse reward creates credit assignment problems: the model receives one scalar number for a long sequence of tokens or actions.

Main Concepts

Reinforcement Learning as Efficient In-Context Learning

Daniel introduced an intuition:

Reinforcement learning is efficient in-context learning.

Suppose we ask a language model:

Create a Python function for fast matrix multiplication.

A pure in-context approach might sample many candidate programs:

$$y^{(1)}, y^{(2)}, \dots, y^{(K)} \sim \pi_{\theta}(\cdot \mid x),$$

test each candidate, score them, and then put the best and worst examples back into the prompt. A second prompt may look like:

Here are previous attempts with scores from -100 to 100 . Produce a better implementation.

This uses context length as the memory mechanism. The model improves within the prompt but does not update its weights. The problem is that long-context learning is expensive and degrades as context grows. Even if a model supports a million tokens, retrieving and using the right information from that context is imperfect.

Reinforcement learning compresses the information from rollouts into the model weights. Instead of storing examples in the context, we use rewards to update θ :

$$\theta \leftarrow \theta + \eta \nabla_{\theta} J(\theta),$$

where

$$J(\theta) = \mathbb{E}_{y \sim \pi_{\theta}(\cdot \mid x)} [R(x, y)].$$

The goal is to increase the probability of high-reward completions and decrease the probability of low-reward completions.

The Token-Level Credit Assignment Problem

A simple reinforcement learning setup may assign the same scalar reward to every token in an output. If a program receives reward -100 , then every token in the program is treated as bad:

$$A_i = -100 \quad \text{for all tokens } i.$$

This is inefficient because some tokens may be correct while only one line causes failure. For example, a generated program may have a correct function signature and correct matrix multiply expression, but one erroneous line:

$$C = AB + 0.001.$$

The true problem is the additive error term, not the entire response. Assigning -100 to every token creates noisy credit assignment.

Process Supervision

Process supervision improves the signal by assigning rewards to intermediate steps, lines, or reasoning actions:

$$R = \sum_{i=1}^T r_i.$$

Instead of one final scalar reward, the model receives a more detailed map of what was good and bad. For code, this could mean:

$$r_i = \begin{cases} +1, & \text{line } i \text{ follows instructions,} \\ 0, & \text{line } i \text{ is neutral,} \\ -10, & \text{line } i \text{ introduces an error.} \end{cases}$$

The lecture notes that process supervision is more informative, but it requires either human labeling or an automated judge. When an LLM is used as a judge, the system becomes vulnerable to judge errors and reward hacking.

Reinforcement Learning Requires Nonzero Probability of Success

A core law stated in the lecture is:

For reinforcement learning to work, the probability of a good answer must be greater than zero.

Mathematically, if G is the set of successful trajectories, then we need:

$$P_{\theta}(\tau \in G) > 0.$$

If

$$P_{\theta}(\tau \in G) = 0,$$

then no amount of sampling will produce reward. The model cannot learn because it never observes success.

This is why pre-training, supervised fine-tuning, priming, formatting examples, and curriculum learning matter. They move the model into a region where success is possible:

$$P_{\theta_{\text{after SFT}}}(\tau \in G) > P_{\theta_{\text{base}}}(\tau \in G).$$

Curriculum Learning

Curriculum learning addresses the nonzero-probability problem by starting with easier tasks and gradually increasing difficulty.

Let task difficulty be d . A curriculum samples tasks from:

$$p_t(d),$$

where early in training $p_t(d)$ emphasizes low difficulty, and later in training it shifts toward harder tasks.

A simple annealed curriculum is:

$$p_t(d = \text{easy}) = \max(0, 1 - \alpha t),$$

$$p_t(d = \text{hard}) = 1 - p_t(d = \text{easy}).$$

The lecture also warns that overly hand-designed curricula can constrain search. A randomized schedule can be more robust:

$$d_t \sim p_t(d),$$

where $p_t(d)$ gradually changes but still samples diverse difficulties.

Reward Hacking

Reward hacking occurs when the model exploits the reward function rather than solving the intended task.

The lecture gave several examples:

- a model deletes the benchmark timer;
- a model edits inputs A or B to make matrix multiplication easier;
- a model caches outputs globally;

- a model imports NumPy or PyTorch when asked to write a raw algorithm;
- an agent exploits a game bug;
- a policy satisfies the literal reward while violating the human intent.

The general failure mode is:

$$\arg \max_{\pi} R_{\text{specified}}(\pi) \neq \arg \max_{\pi} R_{\text{intended}}(\pi).$$

David gave the policy-making analogy: the reward function is like a law. If it has loopholes, search will find them. The search process gives exactly what was specified, not necessarily what was wanted.

Perverse Incentives

A historical analogy from the lecture was the cobra bounty. If a government pays people for dead cobras, people may breed cobras to collect the bounty. The reward is optimized, but the real objective is worsened.

The formal lesson is:

$$\text{Proxy objective} \neq \text{true objective}.$$

A reward function must be designed and audited with adversarial pressure in mind.

Reinforcement Learning Algorithms

Policy Gradient Objective

The standard reinforcement learning objective is:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau)].$$

The policy gradient estimator is:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau) \nabla_{\theta} \log \pi_{\theta}(\tau)].$$

For a language model:

$$\log \pi_{\theta}(y \mid x) = \sum_{i=1}^T \log p_{\theta}(y_i \mid x, y_{<i}).$$

A reward-weighted token update is therefore:

$$\nabla_{\theta} J(\theta) \approx R(x, y) \sum_{i=1}^T \nabla_{\theta} \log p_{\theta}(y_i \mid x, y_{<i}).$$

This explains why assigning a single final reward to all tokens is crude: every token receives the same credit or blame.

Advantage Function

Instead of using raw rewards, modern algorithms use an advantage:

$$A_t = R_t - b_t,$$

where b_t is a baseline, often a value estimate. The advantage says whether an action was better or worse than expected.

$$A_t > 0 \quad \Rightarrow \quad \text{increase probability of the action,}$$

$$A_t < 0 \quad \Rightarrow \quad \text{decrease probability of the action.}$$

PPO

Proximal policy optimization constrains policy updates so that the new policy does not drift too far from the old one. Define the likelihood ratio:

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)}.$$

The clipped PPO objective is:

$$L^{\text{PPO}}(\theta) = \mathbb{E}_t [\min(\rho_t(\theta) A_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) A_t)].$$

PPO often includes a value model and sometimes a reward model. This makes it powerful but expensive.

GRPO

GRPO, or group relative policy optimization, was described as a more efficient alternative that removes the separate value model. Instead of estimating a value function, it compares multiple completions from the same prompt.

For a prompt x , sample G outputs:

$$y^{(1)}, y^{(2)}, \dots, y^{(G)} \sim \pi_{\theta}(\cdot \mid x).$$

Compute rewards:

$$r^{(j)} = R(x, y^{(j)}).$$

A group-normalized advantage can be written as:

$$A^{(j)} = \frac{r^{(j)} - \mu_r}{\sigma_r + \epsilon},$$

where

$$\mu_r = \frac{1}{G} \sum_{j=1}^G r^{(j)}$$

and

$$\sigma_r^2 = \frac{1}{G} \sum_{j=1}^G \left(r^{(j)} - \mu_r \right)^2.$$

The model learns to prefer completions that are better than other completions for the same prompt.

RLVR

RLVR means reinforcement learning with verifiable rewards. Instead of relying on a learned reward model, the system uses a verifier:

$$R(x, y) = V(x, y).$$

Examples include:

- unit tests for code;
- exact-answer checks for math;
- regular expressions for formatting;
- compiler success or failure;
- benchmark speedups;
- game score;

- environment-provided rewards.

RLVR is powerful because it reduces ambiguity. However, it is limited to domains where rewards can be verified reliably.

RLVE

The lecture discussed moving from verifiable rewards to verifiable environments. Instead of static prompt-answer verification, an environment allows multi-step interaction:

$$(o_{t+1}, r_t, d_t) = E(o_t, a_t).$$

This is crucial for agents because real-world tasks are sequential. A model using a browser, debugger, shell, or repository needs to adapt after each observation.

OpenEnv and Environment Standardization

Motivation

OpenEnv is introduced as a community project to make reinforcement learning environments first-class, shareable objects in the same way models and datasets are shareable.

The problem is fragmentation. Environments are often scattered across GitHub repositories, with different APIs, dependency assumptions, launch commands, container formats, and reward schemas.

OpenEnv attempts to standardize:

- how environments are defined;
- how actions and observations are represented;
- how environments are reset and stepped;
- how they are deployed;
- how they are shared;
- how training frameworks communicate with them.

Core Environment Interface

An OpenEnv-style environment exposes three central methods:

```
class Environment:
    def reset(self):
        """Start a fresh episode."""
        ...
```

```

def step(self, action):
    """Apply one action and return observation, reward, and done flag."""
    ...

def state(self):
    """Return the current state or metadata."""
    ...

```

The conceptual interface is:

$$\text{reset} : \emptyset \rightarrow o_0,$$

$$\text{step} : (o_t, a_t) \rightarrow (o_{t+1}, r_t, d_t),$$

$$\text{state} : \emptyset \rightarrow s_t.$$

Actions, Observations, and State

OpenEnv encourages defining explicit data classes for actions, observations, and state:

```

from dataclasses import dataclass

@dataclass
class Action:
    value: str

@dataclass
class Observation:
    message: str
    reward: float
    done: bool

@dataclass
class State:
    metadata: dict

```

The important design principle is that the schema should be clear enough for trainers, evaluators, and agents to use without custom glue code for every environment.

Connect Four Example

For Connect Four:

- **Action:** choose a column.
- **Observation:** board state, legal actions, done flag, reward.
- **State:** internal board and current player.

A simplified environment is:

```
import numpy as np
from dataclasses import dataclass

@dataclass
class ConnectFourAction:
    column: int

@dataclass
class ConnectFourObservation:
    board: list
    legal_actions: list
    done: bool
    reward: float

class ConnectFourEnv:
    def reset(self):
        self.board = np.zeros((6, 7), dtype=int)
        self.player = 1
        return self._observe(0.0, False)

    def step(self, action: ConnectFourAction):
        col = action.column
        if col not in self._legal_actions():
            return self._observe(-1.0, True)

        row = self._drop_piece(col)
        won = self._check_win(row, col)

        if won:
            return self._observe(1.0, True)

        self.player = 3 - self.player
        return self._observe(0.0, False)

    def state(self):
        return {
            "board": self.board.tolist(),
            "player": self.player,
        }
```

Wordle Example

Wordle is a clean RL environment because it is sequential and partially observable. The model guesses a five-letter word, receives feedback, and must adapt.

The environment state includes:

$$h_t = [(g_1, f_1), (g_2, f_2), \dots, (g_t, f_t)],$$

where g_i is a guess and f_i is feedback.

Feedback symbols can be represented as:

$$f_{i,k} = \begin{cases} 2, & \text{letter correct and in correct position,} \\ 1, & \text{letter present but in wrong position,} \\ 0, & \text{letter absent.} \end{cases}$$

A shaped reward might be:

$$R = R_{\text{success}} + \lambda_{\text{green}} R_{\text{green}} + \lambda_{\text{yellow}} R_{\text{yellow}} - \lambda_{\text{invalid}} R_{\text{invalid}}.$$

Here:

- R_{success} rewards solving the word.
- R_{green} rewards keeping known correct letters.
- R_{yellow} rewards using known present letters in new positions.
- R_{invalid} penalizes invalid guesses or format violations.

2048 Example

The 2048 environment has a board:

$$B_t \in \mathbb{N}^{4 \times 4}.$$

Actions are discrete:

$$a_t \in \{0, 1, 2, 3\},$$

corresponding to directions such as up, down, left, and right. A key lecture point was that the model does not necessarily need to be told what each integer means. Through interaction, it can learn the semantics of action IDs.

The reward can be based on score improvement:

$$r_t = \text{score}(B_{t+1}) - \text{score}(B_t).$$

A terminal reward can reward reaching large tiles:

$$R_{\text{tile}} = \log_2 \left(\max_{i,j} B_{i,j} \right).$$

Kernel Sandbox Example

For GPUMODE, an especially relevant environment is a kernel optimization sandbox. The agent proposes CUDA, Triton, or Python code. The environment compiles it, checks correctness, benchmarks speed, and returns reward.

A possible reward is:

$$R = \lambda_{\text{correct}} R_{\text{correct}} + \lambda_{\text{speed}} R_{\text{speed}} - \lambda_{\text{format}} R_{\text{format}} - \lambda_{\text{unsafe}} R_{\text{unsafe}}.$$

If t_{baseline} is the baseline runtime and $t_{\text{candidate}}$ is the candidate runtime, a speedup reward can be:

$$R_{\text{speed}} = \log \left(\frac{t_{\text{baseline}}}{t_{\text{candidate}} + \epsilon} \right).$$

Correctness can be measured by numerical error:

$$\text{err} = \frac{\|C_{\text{candidate}} - C_{\text{reference}}\|_F}{\|C_{\text{reference}}\|_F + \epsilon}.$$

A correctness reward might be:

$$R_{\text{correct}} = \begin{cases} 1, & \text{err} < \tau, \\ -1, & \text{err} \geq \tau. \end{cases}$$

Visual Concept

Draw an LLM on the left emitting kernel code. In the middle, draw a sandbox with compile, run, benchmark, and correctness-check stages. On the right, draw reward components flowing back to the trainer.

Hardware-Level Explanation

Why Environments Are a Systems Problem

Reinforcement learning for agents is not only a machine learning problem. It is a systems problem because each training step may require:

- GPU inference to generate actions;
- CPU execution of environments;
- sandbox startup;
- network communication;
- logging and observability;
- reward computation;
- backpropagation on GPUs.

A rough decomposition of wall-clock time is:

$$T_{\text{step}} = T_{\text{inference}} + T_{\text{environment}} + T_{\text{reward}} + T_{\text{training}} + T_{\text{sync}}.$$

Daniel emphasized that as reinforcement learning progresses, inference can dominate:

$$T_{\text{inference}} \gg T_{\text{training}}.$$

In some regimes:

$$T_{\text{inference}} \approx 0.99T_{\text{total}},$$

which means faster inference is the highest-leverage optimization.

GPU Role in RL Training

The GPU is used for:

- sampling rollouts from the policy;
- computing log probabilities;
- computing losses;
- backpropagating through the model;
- synchronizing policy weights with inference engines;
- serving fast decoding via engines such as vLLM.

The policy update requires storing or recomputing token log probabilities:

$$\log p_{\theta}(y_i \mid x, y_{<i}).$$

For large models, memory is consumed by:

$$M_{\text{total}} = M_{\text{weights}} + M_{\text{optimizer}} + M_{\text{gradients}} + M_{\text{activations}} + M_{\text{KV cache}}.$$

In RL, the KV cache can become especially important because long rollouts increase sequence length.

Long Context and Memory Pressure

The attention cost for a sequence length L and hidden dimension d is approximately:

$$\text{FLOPs}_{\text{attention}} = O(L^2 d).$$

The KV cache memory is:

$$M_{\text{KV}} = 2 \cdot L \cdot n_{\text{layers}} \cdot n_{\text{kv heads}} \cdot d_{\text{head}} \cdot b,$$

where b is bytes per element.

For long-horizon agents, L can grow very large because the model accumulates observations, actions, and reasoning traces. This motivates:

- efficient inference kernels;
- KV cache optimization;
- long-context training methods;
- gradient checkpointing;
- offloading;
- rollout truncation or summarization.

LoRA and QLoRA

Daniel discussed using LoRA and QLoRA to make RL feasible on smaller hardware.

LoRA represents a weight update as a low-rank matrix:

$$W' = W + \Delta W,$$

$$\Delta W = \frac{\alpha}{r} BA,$$

where:

- $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is frozen;
- $A \in \mathbb{R}^{r \times d_{\text{in}}}$;
- $B \in \mathbb{R}^{d_{\text{out}} \times r}$;
- r is the LoRA rank;
- α is a scaling factor.

Only A and B are trained, reducing memory and optimizer state.

QLoRA quantizes the frozen base model to low precision, often 4-bit, while training LoRA adapters. The important deployment warning from the lecture is:

Do not merge LoRA adapters into an upcasted 4-bit model. Merge learned adapters back into the original 16-bit base model.

Conceptually:

$$W_{\text{final}} = W_{\text{fp16 original}} + \Delta W_{\text{LoRA}}.$$

Not:

$$W_{\text{final}} \neq \text{upcast}(W_{\text{int4}}) + \Delta W_{\text{LoRA}}.$$

The latter can damage model accuracy because quantization error becomes baked into the merged weights.

Inference-Training Precision Mismatch

For RL, the model used for generation should match the model being optimized. If training uses QLoRA but inference uses different precision or different adapters, then rollouts are sampled from a different policy than the one being updated.

The desired condition is:

$$\pi_{\text{sample}} = \pi_{\text{train}}.$$

If:

$$\pi_{\text{sample}} \neq \pi_{\text{train}},$$

then the gradient estimate becomes biased because the log probabilities used for the update do not correspond to the distribution that produced the actions.

Sandbox Scaling

Zach’s talk reframed RL environments as a CPU fleet problem. GPUs get attention, but every rollout may need a sandbox. A simplified demand model is:

$$N_{\text{sandboxes}} \approx N_{\text{rollouts}} \times N_{\text{parallel envs}} \times T_{\text{trajectory}}.$$

If trajectories become long and stateful, sandbox demand grows. Unlike a stateless batch request, a long-running environment may need persistent resources.

Hot Path Versus Long Tail

The lecture separates environment scaling into two regimes:

Hot Path

Hot-path environments are well-known, battle-tested, and frequently reused. They can be pre-warmed:

$$T_{\text{startup}}^{\text{hot}} \ll T_{\text{startup}}^{\text{cold}}.$$

The target startup time mentioned was on the order of hundreds of milliseconds.

Long Tail

Long-tail environments are numerous and diverse. They cannot all be pre-warmed. They require:

- common base images;
- layer caching;
- registry peering;
- standardized dependencies;
- container or microVM startup optimization.

For long trajectories, cold start cost can be amortized:

$$T_{\text{amortized}} = \frac{T_{\text{startup}}}{T_{\text{trajectory}}}.$$

If $T_{\text{trajectory}}$ is large, a 10-second startup may be acceptable.

Container Registry Bottleneck

At scale, moving container layers becomes a bottleneck. If each environment requires pulling image layers, total bandwidth demand is:

$$B_{\text{registry}} = \sum_{i=1}^N S_i f_i,$$

where S_i is image size and f_i is pull frequency.

Layer caching reduces this if many images share layers:

$$B_{\text{effective}} = B_{\text{registry}}(1 - h),$$

where h is cache hit rate. Standardizing base images increases h .

MicroVM Snapshots

A microVM can be snapshotted after initialization. Restoring from snapshot can be much faster than booting from scratch.

If:

$$T_{\text{boot}} \gg T_{\text{restore}},$$

then snapshots reduce cold start. However, once restore time becomes very small, network latency may dominate:

$$T_{\text{startup}} \approx T_{\text{network}} + T_{\text{restore}}.$$

Agentic MicroVM Mental Model

The lecture proposed a future direction: an agentic microVM that combines:

- Docker-like authoring convenience;
- microVM isolation;
- fast startup;
- explicit agent-tool primitives;
- safe-by-default execution;
- observability hooks.

The analogy is kernel space versus user space. In normal operating systems, the kernel protects resources and provides abstractions. In agentic execution, the sandbox or microVM should protect the host and provide tool abstractions for agents.

Visual Concept

Draw a microVM boundary. Inside it, place a user-space agent process, files, tools, browser, and compiler. Outside it, place the host scheduler, registry cache, monitoring, and reward service.

Mathematical Reasoning

Pass-at- K Versus Reinforcement Learning

A simple baseline is pass-at- K : sample K completions and choose the best. If each sample succeeds independently with probability p , then:

$$P(\text{success at } K) = 1 - (1 - p)^K.$$

For small p , this is approximately:

$$P(\text{success at } K) \approx Kp.$$

Pass-at- K can work if p is nontrivial and evaluation is cheap. Reinforcement learning attempts to increase p itself:

$$p_{t+1} > p_t.$$

The tradeoff is that RL is more expensive per improvement but can shift the policy distribution for future tasks.

Policy Distribution Shaping

Daniel described RL as changing the probability landscape. If a bad answer receives negative reward, its probability decreases:

$$\pi_{\theta_{t+1}}(y_{\text{bad}} \mid x) < \pi_{\theta_t}(y_{\text{bad}} \mid x).$$

If a good answer receives positive reward, its probability increases:

$$\pi_{\theta_{t+1}}(y_{\text{good}} \mid x) > \pi_{\theta_t}(y_{\text{good}} \mid x).$$

Because probabilities sum to one:

$$\sum_y \pi_\theta(y \mid x) = 1,$$

increasing one region of output space decreases others.

Reward Shaping

Sparse rewards are hard. Reward shaping adds intermediate signals:

$$R_{\text{total}} = R_{\text{task}} + \sum_{k=1}^K \lambda_k R_k.$$

For Wordle:

$$R_{\text{total}} = R_{\text{solve}} + \lambda_g R_{\text{green}} + \lambda_y R_{\text{yellow}} - \lambda_i R_{\text{invalid}}.$$

For code:

$$R_{\text{total}} = R_{\text{tests}} + \lambda_c R_{\text{compile}} + \lambda_s R_{\text{speed}} - \lambda_u R_{\text{unsafe}} - \lambda_f R_{\text{format}}.$$

The challenge is that every term can introduce a new loophole. Reward shaping improves learning but increases specification complexity.

Dynamic Reward Weighting

Ben and Daniel discussed reward functions whose weights change over time. Let:

$$R_t = \sum_{k=1}^K \lambda_k(t) R_k.$$

A formatting reward may be important early but unnecessary later:

$$\lambda_{\text{format}}(t) = \lambda_0 e^{-\alpha t}.$$

A length penalty may be useful early to prevent rambling but harmful later for complex tasks:

$$\lambda_{\text{length}}(t) = \lambda_0 \max\left(0, 1 - \frac{t}{T}\right).$$

Dynamic weighting is another hyperparameter, but it can prevent auxiliary rewards from becoming regressive after the model has learned the basic behavior.

Length Bias

If all successful trajectories receive the same reward, short trajectories may be overweighted because they use fewer tokens or steps. A normalized objective may divide by length:

$$R_{\text{norm}} = \frac{R}{T^\alpha},$$

where α controls the strength of length normalization. However, this can punish legitimate long-horizon reasoning. Long tasks may require larger rewards or staged training.

Inference Cost Dominance

Let N be the number of rollout samples, L the average generated length, and c_{tok} the cost per generated token. Then:

$$T_{\text{inference}} \approx NLc_{\text{tok}}.$$

For RL, increasing group size G , context length L , or number of environments increases inference cost linearly or worse. Therefore, optimizing decoding throughput has direct impact on RL wall-clock time.

Code Walkthrough

Minimal OpenEnv Skeleton

A minimal environment has reset, step, and state:

```
from dataclasses import dataclass

@dataclass
class SimpleAction:
    text: str

@dataclass
class SimpleObservation:
    message: str
    reward: float
    done: bool

class EchoEnv:
    def __init__(self):
        self.turn = 0
```

```

def reset(self):
    self.turn = 0
    return SimpleObservation(
        message="Episode started.",
        reward=0.0,
        done=False,
    )

def step(self, action: SimpleAction):
    self.turn += 1
    reward = 1.0 if "correct" in action.text.lower() else -1.0
    done = self.turn >= 3
    return SimpleObservation(
        message=f"You said: {action.text}",
        reward=reward,
        done=done,
    )

def state(self):
    return {"turn": self.turn}

```

Explanation

The environment is stateful because it stores `self.turn`. The `reset` method starts a fresh episode. The `step` method applies an action and returns a reward. In a real RL training loop, the language model would generate `action.text`; the environment would score it; the trainer would use the reward to update the policy.

Wordle Rollout Logic

A simplified rollout for Wordle:

```

def rollout_wordle(model, tokenizer, env, system_prompt, max_turns=6):
    observation = env.reset()
    messages = [{"role": "system", "content": system_prompt}]
    total_reward = 0.0

    for turn in range(max_turns):
        messages.append({
            "role": "user",
            "content": observation.message,
        })

        prompt = tokenizer.apply_chat_template(
            messages,
            tokenize=False,

```

```

        add_generation_prompt=True,
    )

    output = model.generate_text(prompt)
    guess = extract_guess(output)

    result = env.step({"guess": guess})
    observation = result.observation
    total_reward += result.reward

    messages.append({
        "role": "assistant",
        "content": output,
    })

    if observation.done:
        break

    return {
        "messages": messages,
        "reward": total_reward,
    }

```

Explanation

The rollout accumulates interaction history. Each turn includes the current observation, a model-generated guess, an environment step, and a reward. The model is not only predicting a static answer; it is adapting based on feedback.

The hardware bottleneck is usually model generation, not the Wordle environment. The environment is CPU-light. The model inference may require GPU memory for weights and KV cache.

2048 Strategy Execution With Timeout

A key safety trick is bounding execution time:

```

import signal
from functools import wraps

class TimeoutError(Exception):
    pass

def execute_with_time_limit(seconds):
    def decorator(fn):
        @wraps(fn)
        def wrapper(*args, **kwargs):

```



```

def handler(signum, frame):
    raise TimeoutError("Execution timed out.")

old_handler = signal.signal(signal.SIGALRM, handler)
signal.alarm(seconds)

try:
    return fn(*args, **kwargs)
finally:
    signal.alarm(0)
    signal.signal(signal.SIGALRM, old_handler)

return wrapper
return decorator

```

Explanation

This prevents generated strategies from running forever. For RL environments, infinite loops are common because models may produce repetitive or invalid policies. A timeout converts unbounded execution into a bounded failure that can receive negative reward:

$$R_{\text{timeout}} < 0.$$

Checking Disallowed Imports

A generated strategy may cheat by importing libraries that solve the task. A module checker can reject such code:

```

import ast

ALLOWED_MODULES = {"math", "random", "itertools", "collections"}

def check_python_modules(code: str):
    tree = ast.parse(code)

    for node in ast.walk(tree):
        if isinstance(node, ast.Import):
            for alias in node.names:
                root = alias.name.split(".")[0]
                if root not in ALLOWED_MODULES:
                    return False, f"Disallowed import: {root}"

        if isinstance(node, ast.ImportFrom):
            root = (node.module or "").split(".")[0]
            if root not in ALLOWED_MODULES:

```

```

        return False, f"Disallowed import: {root}"

    return True, "Imports are allowed."

```

Explanation

For a kernel-generation benchmark, importing `torch` or `numpy` may defeat the purpose. For a 2048 environment, importing external services or APIs could also be problematic. Static AST checks are not sufficient for full security, but they are useful reward-hacking guards.

Locking Down Global Variables

A generated function should not rely on hidden global state:

```

def create_locked_function(fn, allowed_globals=None):
    if allowed_globals is None:
        allowed_globals = {
            "__builtins__": {
                "range": range,
                "len": len,
                "min": min,
                "max": max,
                "sum": sum,
                "abs": abs,
            }
        }

    code = fn.__code__
    safe_fn = type(fn)(
        code,
        allowed_globals,
        fn.__name__,
        fn.__defaults__,
        fn.__closure__,
    )
    return safe_fn

```

Explanation

This reduces cheating through global caches or imported modules. In benchmarking environments, hidden global state is dangerous because the model may exploit repeated inputs or cache reference outputs.

Reward Function for Generated Strategy

A simplified reward function for a generated Python strategy:

```

def reward_strategy(code: str, env):
    ok, message = check_python_modules(code)
    if not ok:
        return -20.0

    try:
        fn = compile_strategy(code)
        fn = create_locked_function(fn)
    except Exception:
        return -2.0

    try:
        score = run_strategy_with_timeout(fn, env, seconds=5)
    except TimeoutError:
        return -5.0
    except Exception:
        return -1.0

    if score >= 2048:
        return 20.0
    if score >= 1024:
        return 10.0
    if score >= 512:
        return 5.0

    return 0.5

```

Explanation

The reward has multiple terms:

$$R = R_{\text{imports}} + R_{\text{compilation}} + R_{\text{timeout}} + R_{\text{score}}.$$

The model is heavily penalized for disallowed imports, moderately penalized for invalid code or timeout, and rewarded for actually playing well.

Kernel Optimization Environment

A CUDA-oriented environment might look like:

```

@dataclass
class KernelAction:
    code: str

@dataclass
class KernelObservation:

```

```

compile_ok: bool
tests_ok: bool
runtime_ms: float
error: float
reward: float
message: str

class KernelSandbox:
    def __init__(self, baseline_ms):
        self.baseline_ms = baseline_ms

    def reset(self):
        return {
            "task": "Write a fast matrix multiplication kernel.",
            "baseline_ms": self.baseline_ms,
        }

    def step(self, action: KernelAction):
        compile_ok, binary_or_error = compile_kernel(action.code)
        if not compile_ok:
            return KernelObservation(
                compile_ok=False,
                tests_ok=False,
                runtime_ms=float("inf"),
                error=float("inf"),
                reward=-10.0,
                message=binary_or_error,
            )

        tests_ok, error = run_correctness_tests(binary_or_error)
        if not tests_ok:
            return KernelObservation(
                compile_ok=True,
                tests_ok=False,
                runtime_ms=float("inf"),
                error=error,
                reward=-5.0,
                message="Correctness failed.",
            )

        runtime_ms = benchmark_kernel(binary_or_error)
        speedup = self.baseline_ms / max(runtime_ms, 1.0e-6)
        reward = math.log(speedup)

        return KernelObservation(
            compile_ok=True,

```

```

        tests_ok=True,
        runtime_ms=runtime_ms,
        error=error,
        reward=reward,
        message="Kernel succeeded.",
    )

```

Hardware Explanation

Each candidate kernel must be compiled and run in a sandbox. Correctness checks compare GPU output against a reference. Benchmarking should use CUDA events and warmup iterations to avoid measuring compilation or first-run overhead.

Important GPU considerations include:

- whether global memory reads are coalesced;
- whether shared memory tiling is used;
- whether tensor cores are used for FP16 or BF16;
- whether register pressure reduces occupancy;
- whether block dimensions map cleanly to warps;
- whether synchronization is correct;
- whether boundary checks cause divergence.

Performance Intuition

Why RL Is Slow

RL is slow because it repeatedly samples, evaluates, and updates. Compared with supervised fine-tuning, the information density per token is lower.

For supervised fine-tuning, each token has a target:

$$\mathcal{L}_{\text{SFT}} = - \sum_{i=1}^T \log p_{\theta}(y_i^* \mid x, y_{<i}^*).$$

For RL, a long sequence may only have one reward:

$$\mathcal{L}_{\text{RL}} = -R \sum_{i=1}^T \log p_{\theta}(y_i \mid x, y_{<i}).$$

This is noisier and less sample-efficient.

Why RL Still Works

RL works because it changes the sampling distribution. Even crude rewards can gradually suppress bad modes and amplify good modes. If the system samples enough rollouts and obtains enough nonzero reward, the policy improves.

The lecture’s phrase was:

Patience is all you need.

This should not be interpreted as saying compute is free. It means the method can discover behavior through search, but the systems stack must make that search affordable.

Inference Throughput Is Critical

For language-model RL, every optimization step may require many generations. If we use group size G , batch size B , and average output length L , generated tokens per step are:

$$N_{\text{tokens}} = BGL.$$

If decoding throughput is q tokens per second, then:

$$T_{\text{decode}} = \frac{BGL}{q}.$$

Increasing q through better inference engines, quantization, batching, and KV cache management directly improves RL iteration speed.

Environment Throughput

For simple games such as Wordle or 2048, the environment may be CPU-cheap. For code environments, browser environments, or repository environments, environment time can dominate.

Let:

$$T_{\text{env}} = T_{\text{startup}} + T_{\text{step}} + T_{\text{verification}}.$$

For code tasks:

$$T_{\text{verification}} = T_{\text{compile}} + T_{\text{test}} + T_{\text{benchmark}}.$$

If T_{compile} is large, batching and caching become important. If $T_{\text{benchmark}}$ is noisy, the reward becomes noisy.

Reward Noise

If measured runtime has noise:

$$\hat{t} = t + \epsilon,$$

then reward based on speedup is noisy:

$$\hat{R} = \log \left(\frac{t_{\text{baseline}}}{t + \epsilon} \right).$$

This can mislead RL. Kernel benchmarking environments need repeated measurements:

$$\bar{t} = \frac{1}{n} \sum_{i=1}^n t_i,$$

and possibly robust statistics such as median runtime.

Sandbox Cost Amortization

For long tasks, startup cost is amortized:

$$\text{overhead fraction} = \frac{T_{\text{startup}}}{T_{\text{startup}} + T_{\text{trajectory}}}.$$

If $T_{\text{trajectory}}$ is large, optimizing startup matters less than throughput, reliability, and safety. If tasks are short, startup dominates.

Scaling With WebSockets

Ben described WebSockets as a way to support multiple concurrent sessions inside one container. Instead of one container per environment instance, a container can host many sessions:

$$N_{\text{sessions/container}} > 1.$$

This improves resource utilization for lightweight environments. It is less suitable for sandboxed code execution where isolation per episode matters.

Common Misconceptions

Misconception: RL Removes the Need for Data Preparation

RL does not remove data work. It changes the nature of data work. Instead of only preparing demonstrations, the researcher must prepare:

- prompts;
- task distributions;
- reward functions;
- environment definitions;
- curricula;
- safety checks;
- logging and inspection pipelines.

Misconception: A Higher Reward Always Means a Better Model

A higher reward can mean:

- the model improved;
- the reward function was hacked;
- the environment has a bug;
- the judge is being exploited;
- the model found a shortcut;
- the metric does not match human intent.

Reward must be interpreted alongside rollout samples, length distributions, failure modes, and independent evaluations.

Misconception: Reward Hacking Means the Model Is Superintelligent

The lecture emphasized that reward hacking often means the reward or environment was underspecified. It is not necessarily evidence of deep intelligence. Search finds loopholes because loopholes are part of the optimization landscape.

Misconception: One Reward Function Is Enough

Single reward functions are fragile. Multiple independent reward functions reduce the chance that one loophole dominates:

$$R = \sum_{k=1}^K \lambda_k R_k.$$

However, multiple rewards also increase complexity and require careful weighting.

Misconception: Long Context Solves RL

Long context helps, but it is not a replacement for learning. In-context examples consume tokens and can become hard to retrieve. RL writes successful behavior into the model weights.

Misconception: Containers Alone Are Sufficient for Future Agents

Containers are convenient, but future agentic systems may need stronger isolation, faster startup, and better primitives. MicroVM-style isolation may become more important as agents execute increasingly autonomous actions.

Misconception: LoRA Is Only for SFT

The lecture discussed evidence that LoRA and QLoRA can work for reinforcement learning when configured properly. Important details include targeting enough modules, choosing good scaling, matching inference and training precision, and merging adapters correctly.

Practical Takeaways

For Training RL Models

- Ensure the model has nonzero probability of success before running expensive RL.
- Use SFT, prompt priming, or curriculum learning when tasks are too hard.
- Prefer verifiable rewards when available.
- Use multiple independent reward functions.
- Inspect rollouts frequently.
- Track rollout length, reward components, invalid-action rates, timeout rates, and formatting failures.
- Use dynamic reward weights when auxiliary rewards become harmful later.
- Be cautious with LLM-as-judge because the judge can be wrong or exploitable.

For Building Environments

- Define clear action, observation, and state schemas.
- Keep reset and step semantics simple.
- Avoid exposing hidden state that makes reward hacking easy.
- Add timeouts for generated code.
- Lock down imports and global variables.
- Use Docker or microVM isolation for untrusted execution.
- Log enough information to reconstruct failures.
- Make environments versioned and reproducible.

For GPU Systems Engineers

- RL performance is often inference-bound.
- Long contexts increase KV cache pressure.
- Fast decoding and efficient batching matter enormously.
- Weight sharing between training and inference engines reduces memory overhead.
- Quantization can enable small-hardware experiments but must be handled carefully.
- Kernel-generation environments need robust correctness and benchmarking.
- Sandbox startup, registry caching, and CPU fleet scaling can become bottlenecks.

For Agent Infrastructure

- Treat environments as first-class artifacts.
- Standardize APIs so trainers can swap environments easily.
- Use providers for local Docker, local Python, Docker Swarm, Slurm, Kubernetes, or remote spaces.
- Separate trusted, semi-trusted, and untrusted workloads.
- Expect future agent systems to need safe-by-default execution.

Project or Experiment Ideas

Experiment 1: Pass-at- K Versus RL

Take a simple code-generation task with unit tests. Compare:

- pass-at- K sampling from a base model;
- SFT on successful samples;
- RL with unit-test reward.

Measure:

$$P(\text{success}), \quad T_{\text{compute}}, \quad \text{tokens generated}, \quad \text{wall-clock time}.$$

Experiment 2: Reward Hacking in Matrix Multiplication

Build an environment that asks a model to implement matrix multiplication. Add reward for correctness and speed. Observe whether the model:

- imports NumPy;
- calls PyTorch;
- caches outputs;
- modifies inputs;
- returns approximate constants;
- deletes timing code.

Then add guards and measure how behavior changes.

Experiment 3: Wordle RL With Reward Shaping

Train a small model on Wordle with different reward functions:

$$R_1 = R_{\text{solve}},$$

$$R_2 = R_{\text{solve}} + \lambda_g R_{\text{green}},$$

$$R_3 = R_{\text{solve}} + \lambda_g R_{\text{green}} + \lambda_y R_{\text{yellow}}.$$

Compare learning speed and failure modes.

Experiment 4: 2048 Strategy Generation

Use a 2048 environment where the model generates Python strategies. Add:

- timeout penalties;
- import restrictions;
- score rewards;

- tile rewards;
- repetition penalties.

Track whether the model learns meaningful board heuristics.

Experiment 5: CUDA Kernel Sandbox

Create an environment for a simple CUDA operation such as vector addition or matrix transpose. Reward correctness and speedup.

Important measurements:

$$\text{bandwidth} = \frac{\text{bytes read} + \text{bytes written}}{t},$$

$$\text{speedup} = \frac{t_{\text{baseline}}}{t_{\text{candidate}}}.$$

Use Nsight Compute to inspect memory coalescing, occupancy, and achieved bandwidth.

Experiment 6: Environment Scaling

Deploy a lightweight environment with:

- one HTTP session per container;
- multiple WebSocket sessions per container;
- multiple containers behind a load balancer.

Measure throughput:

$$Q = \frac{N_{\text{requests}}}{T_{\text{wall}}}.$$

Measure tail latency:

$$p_{95}, \quad p_{99}.$$

Experiment 7: LoRA Versus QLoRA for RL

Train the same small RL task using:

- full fine-tuning;
- LoRA;
- QLoRA.

Compare reward curves, final performance, memory usage, and inference-training mismatch.

Final Mental Model

Reinforcement learning for language-model agents is best understood as search plus memory compression. The model samples behavior, the environment scores it, and gradient descent compresses the lessons from many rollouts into the model weights.

The simplest in-context approach says:

Keep examples in the prompt.

Reinforcement learning says:

Use rewards to move the probability distribution so the model becomes more likely to produce good behavior without needing all previous examples in context.

The entire agentic RL stack can be summarized as:

Model + Environment + Reward + Sandbox + Trainer + Observability
--

Each part can fail:

- If the model cannot ever succeed, RL has no signal.
- If the environment is underspecified, the model finds loopholes.
- If the reward is too sparse, learning is slow.
- If the sandbox is unsafe, generated code can corrupt the system.
- If inference is slow, RL becomes economically infeasible.
- If observability is weak, reward hacking can look like progress.

The lecture's deepest lesson is that future agent training is not just about better algorithms. It is about building reliable worlds for models to practice in. Those worlds need verifiable rewards, scalable execution, safe isolation, and enough diversity to smooth out the jagged intelligence of current models.